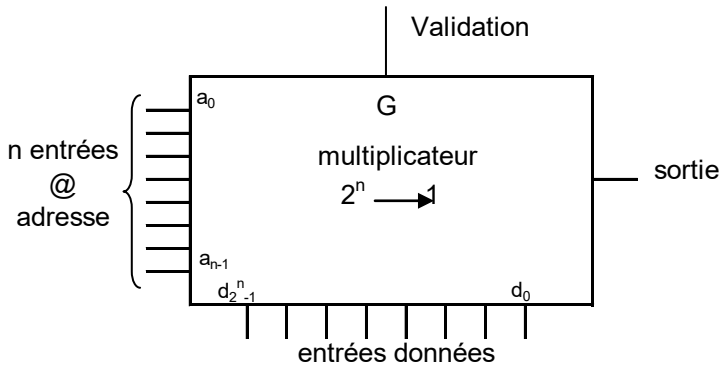
CHAP 4 : CIRCUITS LOGIQUES BINAIRES-CIRCUITS ARITHMÉTIQUES

I. Circuits logiques binaires :

Pour réaliser un circuit logique binaire, le concepteur doit utiliser plusieurs portes logiques élémentaires. Pour faciliter sa tâche, les fabricants fournissent des circuits sous forme intégrée comportant chacun plusieurs portes à des degrés d'intégration différents.
Dans la présente section, nous nous proposons d'étudier plusieurs dispositifs logiques combinatoires couramment utilisés dans les systèmes numériques. Ces dispositifs sont : les multiplexeurs, démultiplexeurs et décodeurs.

A. Multiplexeurs :

Définition :



Un multiplexeur (MUX) est un circuit logique qui possède 2^N entrées d'informations (d_i), N entrées de sélection (a_i) et une sortie unique Y.
Sa fonction consiste à effectuer l'aiguillage de l'une des entrées d'information vers la sortie en fonction du code d'adresse appliqué sur les entrées de sélection.
On pourra de plus trouver une entrée de validation G. Si cette broche n'est pas validée, la sortie Y est égale à '1' (ou '0'), et ce quelle que soit l'adresse appliquée et le niveau des entrées a_i, par contre quand cette broche est validée, le multiplexeur délivre sur sa sortie Y l'état de l'entrée adressée. On l'appelle aussi aiguilleur ou achemineur ;
La représentation fonctionnelle du multiplexeur (par exemple 8 vers 1) est alors donnée par la figure ci-dessous :

- Si G = 1, multiplexeur non validé.
  $\forall d_i, \forall a_i$ y = 1
- Si G = 0, multiplexeur validé.
  $(a_{n-1}.....a_0)_{(2)} = k$ en décimal

$$Y = d_K$$

Représentation par T.V sous une autre forme :

1) $a \oplus b$

a	b	$F = \overline{a}.b + a.\overline{b}$
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

$F = \overline{a}.b.1 + a.\overline{b}.1$

b	F
0	a
1	$\overline{a}$

$F = a$ si b = 0
$F = \overline{a}$ si b = 1
$\rightarrow F = \overline{b}.a + b.\overline{a}$

2)

a	B	S
0	0	c
0	1	1
1	0	0
1	1	$\overline{c}$

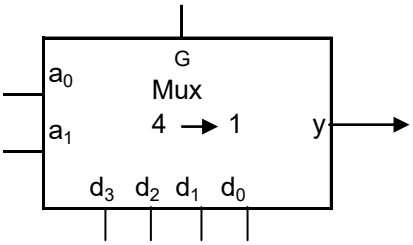
$S = \overline{a}.\overline{b}.c + \overline{a}.b + a.b.\overline{c}$

3) S une fonction de a, b et c
Si a = 0, S = 1
Si a = 1, S = b + c

a	b	C	**S**
0	x	x	**1**
1	0	0	**0**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**

$S = \overline{a} + a.(b + c)$

Exemple : Réalisation d'un Multiplexeur 4 vers 1
1) Table de vérité
2) Expression logique
3) Logigramme



Solution :

1)

G	a_1	a_0	**Y**
1	X	X	**1**
0	0	0	d_0
0	0	1	d_1
0	1	0	d_2
0	1	1	d_3

2) $Y = G + \overline{G}.(\overline{a}_1.\overline{a}_0.d_0 + \overline{a}_1.a_0.d_1 + a_1.\overline{a}_0.d_2 + a_1.a_0.d_3)$

$\rightarrow Y = G + \overline{a}_1.\overline{a}_0.d_0 + \overline{a}_1.a_0.d_1 + a_1.\overline{a}_0.d_2 + a_1.a_0.d_3$

3) Logigramme

Application des Multiplexeurs :

Les applications des multiplexeurs dans le domaine des techniques numériques sont variées. Nous citrons les deux applications suivantes :

- Transmissions : mise en série des données provenant de plusieurs sources (Conversion parallèle-série).
- Génération des fonctions logiques.

Génération d'une fonction logique :

Il est possible d'utiliser les multiplexeurs pour matérialiser une fonction logique à partir d'une table de vérité, sans devoir passer par le processus de simplification.

Quand on utilise un multiplexeur à cette fin, les entrées de sélection reçoivent les variables logiques, et chaque entrée est raccordée en permanence à 0 ou à 1 suivant la fonction à réaliser.

Exemple : Multiplexeur $4 \rightarrow 1$

$$Y = \overline{a}_1.\overline{a}_0.d_0 + \overline{a}_1.a_0.d_1 + a_1.\overline{a}_0.d_2 + a_1.a_0.d_3$$
$$= m_0.d_0 + m_1.d_1 + m_2.d_2 + m_3.d_3$$

Cas général : Multiplexeur $2^n \rightarrow 1$

$$Y = m_0.d_0 + m_1.d_1 + ... + m_{2^n-1}.d_{2^n-1}$$
$$\Rightarrow Y = \sum_{K=0}^{2^n-1} m_K.d_K$$

Application :

$F(a,b,c) = \overline{a} + \overline{b}.\overline{c}$

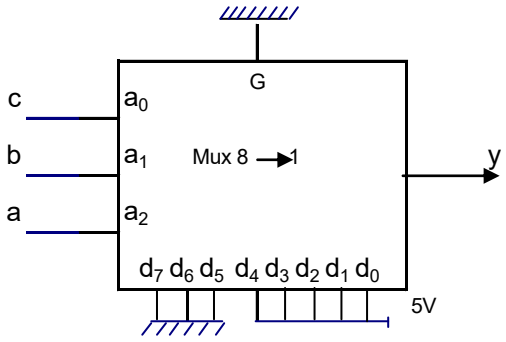
1) Réaliser cette fonction à l'aide d'un Multiplexeur $8 \rightarrow 1$

$$F(a,b,c) = \overline{a}.(b+\overline{b}).(c+\overline{c}) + (a+\overline{a})\overline{b}.\overline{c}$$
$$= \overline{a}.b.c + \overline{a}.b.\overline{c} + \overline{a}.\overline{b}.c + \overline{a}.\overline{b}.\overline{c} + a.\overline{b}.\overline{c} + \overline{a}.\overline{b}.\overline{c}$$
$$F(a,b,c) = \overline{a}.\overline{b}.\overline{c} + \overline{a}.\overline{b}.c + \overline{a}.b.\overline{c} + \overline{a}.b.c + a.\overline{b}.\overline{c}$$

F = R(0, 1, 2, 3, 4)

→ Multiplexeur 8 vers 1 avec a, b, c (variables de la fonction) qui seront les adresses du Multiplexeur et G est fixé au '0'.

On fixe : $d_0 = 1$, $d_1 = 1$, $d_2 = 1$, $d_3 = 1$ et $d_4 = 1$
Et on fixe : $d_5 = d_6 = d_7 = 0$



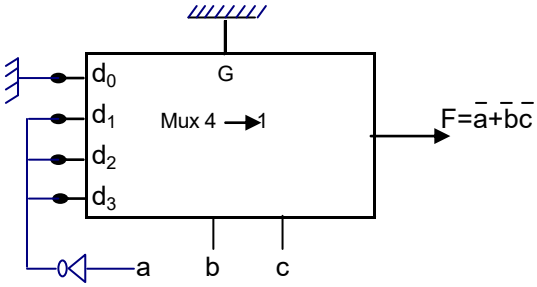
2) Réaliser cette fonction à l'aide d'un Multiplexeur $4 \rightarrow 1$

$F(a,b,c) = \overline{a} + \overline{b}.\overline{c}$ Avec a : poids fort (P_F) et c : poids faible (P_f)

Multiplexeur $4 \rightarrow 1$ $\Rightarrow\Rightarrow\Rightarrow$ $m_0 - m_1 - m_2 - m_3$

$\overline{b}.\overline{c}$ - $\overline{b}.c$ - $b.\overline{c}$ - $b.c$

$$F(a,b,c) = \overline{a}.(b+\overline{b}).(c+\overline{c}) + \overline{b}.\overline{c}$$
$$= \overline{a}.[b.c + \overline{b}.c + b.\overline{c} + \overline{b}.\overline{c}] + \overline{b}.\overline{c}$$
$$= \overline{a}.m_3 + \overline{a}.m_1 + \overline{a}.m_2 + m_0.(\overline{a}+1)$$



Application :

$Z = D.C.\overline{B}.\overline{A} + D.\overline{B}.A + \overline{D}.\overline{B}.\overline{A} + C.B$

1) Multiplexeur $16 \rightarrow 1$
2) Multiplexeur $4 \rightarrow 1$
3) Multiplexeur $8 \rightarrow 1$

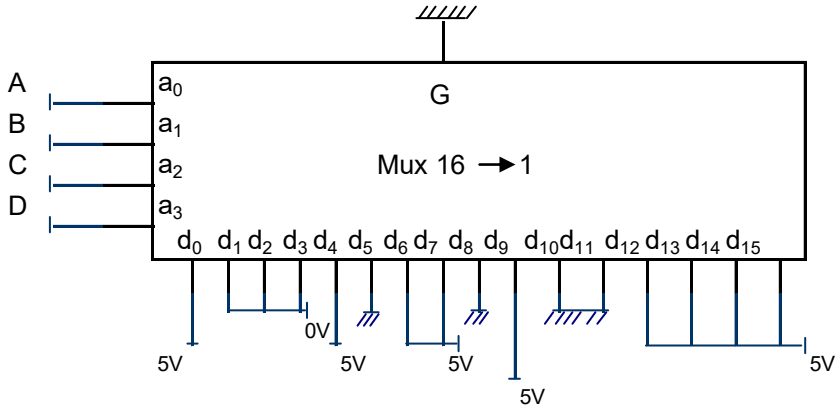
Solution :

1) $Z = D.C.\overline{B}.\overline{A} + D.(C+\overline{C}).\overline{B}.A + \overline{D}.(C+\overline{C}).\overline{B}.\overline{A} + (D+\overline{D}).C.B.(A+\overline{A})$

$= D.C\overline{B}.\overline{A} + D.C.\overline{B}.A + D.\overline{C}.\overline{B}.A + \overline{D}.C.\overline{B}.\overline{A} + \overline{D}.\overline{C}.\overline{B}.\overline{A} + D.C.B.A + D.C.B.\overline{A} + \overline{D}.C.B.A + \overline{D}.C.B.\overline{A}$

= d_{12} + d_{13} + d_9 + d_4 + d_0 + d_{15} + d_{14} + d_7 + d_6

= R(0, 4, 6, 7, 9, 12, 13, 14, 15)



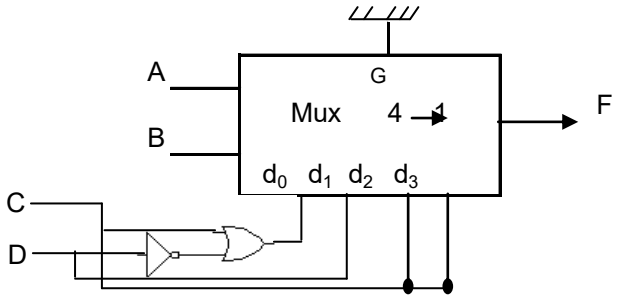
2) $Z(D,C,B,A) = D.C.\overline{B}.\overline{A} + D.\overline{B}.\overline{A} + \overline{D}.\overline{B}.\overline{A} + C.B$

$= D.C.m_0 + D.m_1 + \overline{D}.m_0 + C.B.A + C.B.\overline{A}$

$= D.C.m_0 + D.m_1 + \overline{D}.m_0 + C.m_3 + C.m_2$

$= m_0.(D.C + \overline{D}) + m_1.D + m_2.C + m_3.C$

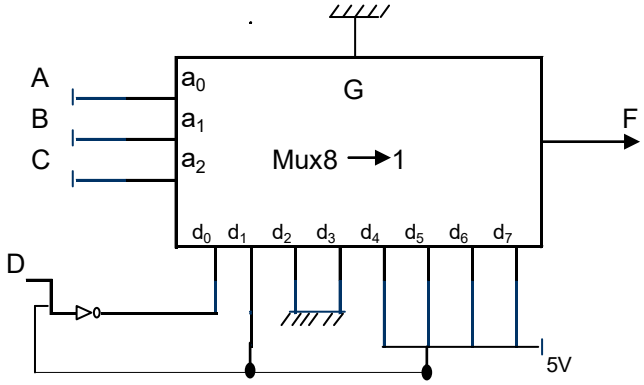
$= m_0.(\overline{D} + C) + m_1.D + m_2.C + m_3.C$



3) $Z(D,C,B,A) = D.C.\overline{B}.\overline{A} + D.\overline{B}.A + \overline{D}.\overline{B}.\overline{A} + C.B$

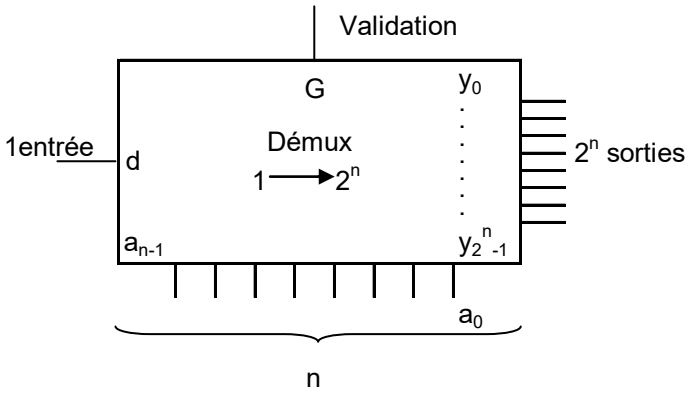
$= D.m_4 + D.m_5 + D.m_1 + \overline{D}.m_4 + \overline{D}.m_0 + m_7 + m_6$

$= m_0.\overline{D} + m_1.D + m_4 + m_5.D + m_6 + m_7$



B. Démultiplexeur :

Un démultiplexeur (DEMUX) est un circuit logique qui possède une seule entrée d et 2^n voies de sortie. Il transmet les données d'entrée (données séries) vers une des 2^n voies de sortie selon l'adresse du code appliqué sur les N entrées de sélection, il fonctionne comme un commutateur.
Les données qui parviennent en série à l'entrée du démultiplexeur seront fournies en parallèle en sortie.



. Si G = 1 → Démultiplexeur non validé

$\forall a_i$, $\forall d$, $Y_j = 1$ pour tout j

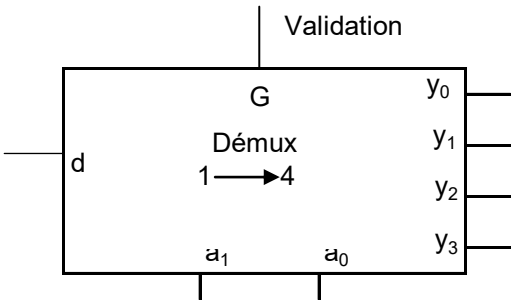
. Si G = 0 $\rightarrow$ Démultiplexeur validé

$(a_{i-1}.....a_0)_{(2)} = k$ en décimal

$Y_K = d$ et $Y_j = 1$ pour $j \neq k$

<u>Application :</u> Démultiplexeur $1 \rightarrow 4$

T.V et les expressions des sorties



1) <u>T.V</u>

G	a_1	a_0	Y_3	Y_2	Y_1	Y_0
1	x	x	**1**	**1**	**1**	**1**
0	0	0	**1**	**1**	**1**	**d**
0	0	1	**1**	**1**	**d**	**1**
0	1	0	**1**	**d**	**1**	**1**
0	1	1	**d**	**1**	**1**	**1**

2) Expressions logiques des sorties :

$$Y_3 = G + \overline{G}.(\overline{a}_1.\overline{a}_0 + \overline{a}_1.a_0 + a_1.\overline{a}_0 + a_1.a_0.d) = G + a_1 + a_0 + d$$

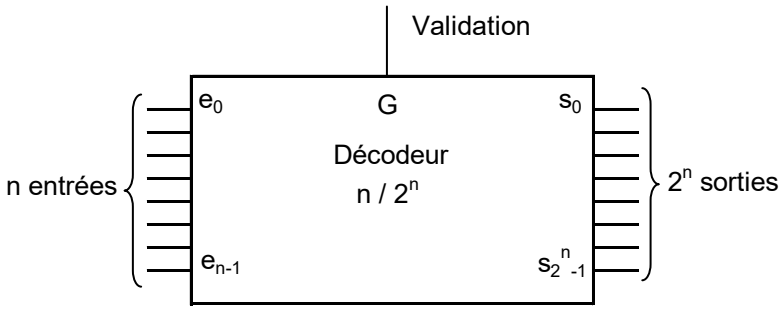
$$Y_2 = G + \overline{G}.(\overline{a}_1.\overline{a}_0 + \overline{a}_1.a_0 + a_1.\overline{a}_0.d + a_1.a_0) = G + \overline{a}_1 + a_0 + d$$

$$Y_1 = G + \overline{G}.(\overline{a}_1.\overline{a}_0 + \overline{a}_1.a_0.d + a_1.\overline{a}_0 + a_1.a_0) = G + a_1 + \overline{a}_0 + d$$

$$Y_0 = G + \overline{G}.(\overline{a}_1.\overline{a}_0.d + \overline{a}_1.a_0 + a_1.\overline{a}_0 + a_1.a_0) = G + \overline{a}_1 + \overline{a}_0 + d$$

C. <u>Décodeur :</u>

Un décodeur est un circuit numérique qui possède N entrées et 2^N sorties. Pour chacune des combinaisons possibles des entrées, seule une ligne de sortie est validée (soit à niveau Haut, soit à niveau Bas selon le type du décodeur).
Les décodeurs sont souvent dotés d'une ou plusieurs entrées de validation G qui servent à valider son fonctionnement.



. Si G = 1 $\rightarrow$ Décodeur non validé

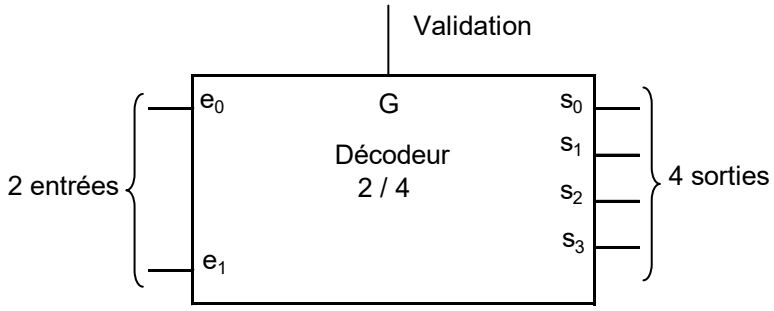
$\forall e_i, S_j = 1$

. Si G = 0 $\rightarrow$ Décodeur validé

$(e_{n-1}.....e_0) = k$: en décimal

$S_K = 0$ et $S_j = 1 \ \ \forall k \neq j$

<u>Exemple :</u> Décodeur 2/4



1) Table de vérité

G	e_1	e_0	S_3	S_2	S_1	S_0
1	x	x	**1**	**1**	**1**	**1**
0	0	0	**1**	**1**	**1**	**0**
0	0	1	**1**	**1**	**0**	**1**
0	1	0	**1**	**0**	**1**	**1**
0	1	1	**0**	**1**	**1**	**1**

2) Expression logique des sorties :

$S_0 = G + e_1 + e_0 = \overline{\overline{G}.\overline{e}_1.\overline{e}_0}$

$S_1 = G + e_1 + \overline{e}_0 = \overline{\overline{G}.\overline{e}_1.e_0}$

$S_2 = G + \overline{e}_1 + e_0 = \overline{\overline{G}.e_1.\overline{e}_0}$

$S_3 = G + \overline{e}_1 + \overline{e}_0 = \overline{\overline{G}.e_1.e_0}$

<u>Application des décodeurs :</u>

. Circuit de sélection : Décodage d'adresse dans les systèmes à microprocesseur.

. Réalisation des fonctions booléennes :

Si G = 0, les sorties sont les Maxtermes relatifs aux entrées et peuvent s'écrire en fonction des mintermes.

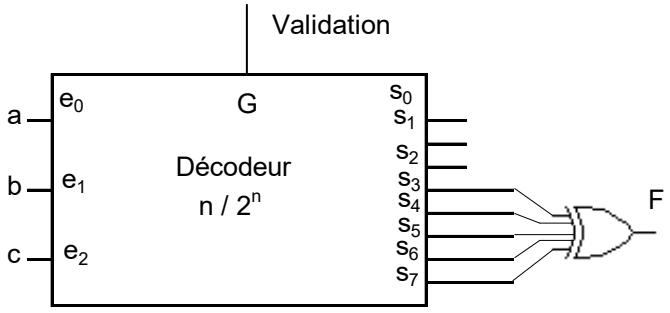
$$S_0 = G + e_1 + e_0 = G + M_3 = G + \overline{m}_0$$
$$S_1 = G + e_1 + \overline{e}_0 = G + M_2 = G + \overline{m}_1$$
$$S_2 = G + \overline{e}_1 + e_0 = G + M_1 = G + \overline{m}_2$$
$$S_3 = G + \overline{e}_1 + \overline{e}_0 = G + M_0 = G + \overline{m}_3$$
$$\overline{M}_j = m_i \qquad \text{j = CR(i) où i = CR(j)}$$

<u>Cas général:</u> Décodeur n/2^n

Pour k = 0, 1 ……. $2^n - 1$

$$S_K = G + M_{CR(K)} = G + \overline{m}_K$$

Exercice:

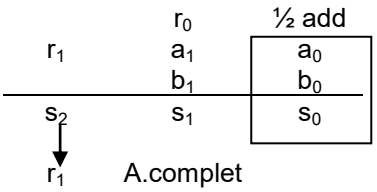


II. <u>Circuits Arithmétiques:</u>

A. <u>Additionneur:</u>

1) <u>Addition de deux nombres binaires à 2 bits:</u>

A (a_1 a_0)
B (b_1 b_0)



½ Additionneur : car il fait la sommation des bits les plus faibles a_0 et b_0.

a) Demi-additionneur (Half Adder : HA):

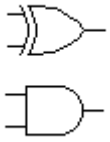
C'est un circuit qui fournit la somme modulo 2 et la retenue de deux chiffres binaires. Appelons A_i, B_i les deux variables d'entrée représentant les bits à additionner, S_i la somme et R_i la retenue (C : appelée ''carry'' en anglais).
La table de vérité du demi-additionneur est donnée par le tableau ci-dessous :

A_i	B_i	S_i	R_i
0	0	**0**	**0**
0	1	**1**	**0**
1	0	**1**	**0**
1	1	**0**	**1**

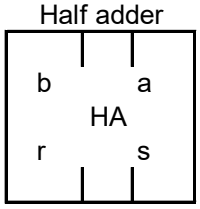
On déduit alors les expressions logiques de S_i et de R_i.

$$S_i = A_i.\overline{B}_i + \overline{A}_i.B_i = A_i \oplus B_i$$
$$R_i = A_i.B_i$$

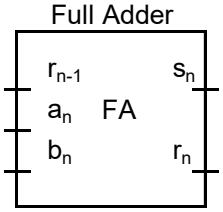
Le circuit logique du demi-additionneur est donné par la figure ci-dessous :



Le demi-additionneur ne peut faire que l'addition des deux chiffres de plus faible poids, puisqu'il ne peut pas prendre en compte la retenue qui provient d'une addition précédente. Pour résoudre cette difficulté, on utilise un circuit à trois entrées : c'est l'additionneur complet (A.C).



b) Additionneur complet : (A.C)



C'est un circuit qui fournit la somme et la retenue de deux chiffres binaires et de la retenue du rang précédent.
Pour cela, l'additionneur complet est un circuit à trois entrées A_i, B_i et R_{i-1} (report précédent) et possédant toujours deux sorties somme S_i et retenue R_i .

La table de vérité de l'additionneur complet est donnée par le tableau ci-dessous :

A_i	B_i	R_{i-1}	S_i	R_i
0	0	0	**0**	**0**
0	0	1	**1**	**0**
0	1	0	**1**	**0**
0	1	1	**0**	**1**
1	0	0	**1**	**0**
1	0	1	**0**	**1**
1	1	0	**0**	**1**
1	1	1	**1**	**1**

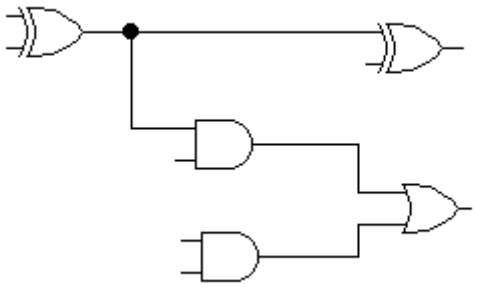
L'expression de la somme S_i est alors :

$$S_i = \overline{A}_i.\overline{B}_i.R_{i-1} + \overline{A}_i.B_i.\overline{R}_{i-1} + A_i.\overline{B}_i.\overline{R}_{i-1} + A_i.B_i.R_{i-1}$$
$$= \overline{A}_i.(B_i.\overline{R}_{i-1} + \overline{B}_i.R_{i-1}) + A_i.(\overline{B}_i.\overline{R}_{i-1} + B_i.R_{i-1})$$
$$= \overline{A}_i.(B_i \oplus R_{i-1}) + A_i.\overline{(B_i \oplus R_{i-1})} = A_i \oplus B_i \oplus R_{i-1}$$

L'expression de la retenue R_i est :

$$R_i = \overline{A}_i.B_i.R_{i-1} + A_i.\overline{B}_i.R_{i-1} + A_i.B_i.\overline{R}_{i-1} + A_i.B_i.R_{i-1}$$
$$= (\overline{A}_i.B_i + A_i.\overline{B}_i).R_{i-1} + A_i.B_i = (A_i \oplus B_i).R_{i-1} + A_i.B_i$$

Ce qui conduit au schéma de la figure ci-dessous relatif à l'additionneur complet.

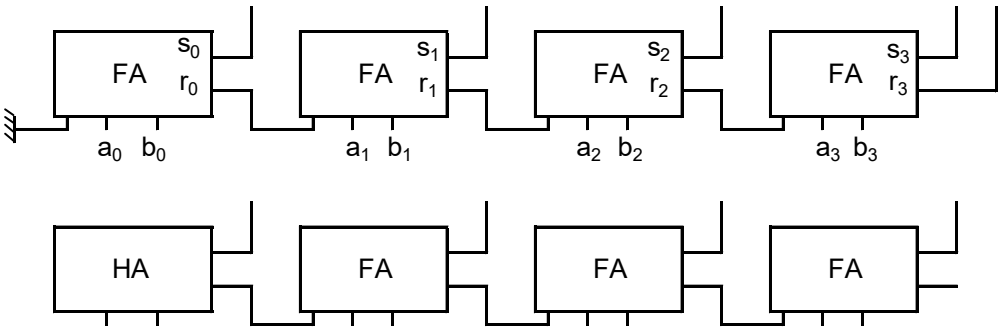


2) <u>Réalisation d'un additionneur à 4 bits à l'aide des additionneurs complets (Full Adder) :</u>

$A(a_3, a_2, a_1, a_0)$

$B(b_3, b_2, b_1, b_0)$

A l'aide des HA et FA



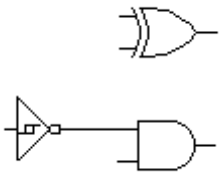
B. Soustracteur :

La soustraction se résume à une opération d'addition, si on exprime les nombres négatifs selon la notation en complément à 2.

A – B = A + CV(B) : voir TP

a	b	**S**	**Emp (Empreinte)**
0	0	**0**	**0**
0	1	**1**	**1**
1	0	**1**	**0**
1	1	**0**	**0**

$$S = a \oplus b$$
$$Emp = \overline{a}.b$$



C. Multiplieur :

Le processus de la multiplication est illustré par un exemple de multiplication de deux nombres binaires à 2 bits suivant :

A (a_1 a_0)
B (b_1 b_0)
P (P_3 P_2 P_1 P_0)

2éme méthode:

		a_1	a_0
		b_1	b_0
		a_1b_0	a_0b_0
	a_1b_1	a_0b_1	
P_3	P_2	P_1	P_0
			⏟
			a_0b_0

1ère méthode : Réalisation à l'aide de portes logiques

∗) **T.V**

A		B		P_3	P_2	P_1	P_0
a_1	a_0	b_1	b_0				
0	0	0	0	**0**	**0**	**0**	**0**
		0	1	**0**	**0**	**0**	**0**
		1	0	**0**	**0**	**0**	**0**
		1	1	**0**	**0**	**0**	**0**
0	1	0	0	**0**	**0**	**0**	**0**
		0	1	**0**	**0**	**0**	**1**
		1	0	**0**	**0**	**1**	**0**
		1	1	**0**	**0**	**1**	**1**
1	0	0	0	**0**	**0**	**0**	**0**
		0	1	**0**	**0**	**1**	**0**
		1	0	**0**	**1**	**0**	**0**
		1	1	**0**	**1**	**1**	**0**
1	1	0	0	**0**	**0**	**0**	**0**
		0	1	**0**	**0**	**1**	**1**
		1	0	**0**	**1**	**1**	**0**
		1	1	**1**	**0**	**0**	**1**

a_1	a_0	b_1	b_0	P_3	P_2	P_1	P_0
0	0	x	X	**0**	**0**	**0**	**0**
0	1	α	β	**0**	**0**	**α**	**β**
1	0	α	B	**0**	**α**	**β**	**0**
1	1	0	0	**0**	**0**	**0**	**0**
1	1	0	1	**0**	**0**	**1**	**1**
1	1	1	0	**0**	**1**	**1**	**0**
1	1	1	1	**1**	**0**	**0**	**1**

∗ **) P_0**

a_1a_0 / b_1b_0	00	01	11	10
00	**0**	**0**	**0**	**0**
01	**0**	**1**	**1**	**0**
11	**0**	**1**	**1**	**0**
10	**0**	**0**	**0**	**0**

$\Rightarrow P_0 = a_0.b_0$

∗ **) P_1**

a_1a_0 / b_1b_0	00	01	11	10
00	**0**	**0**	**0**	**0**
01	**0**	**0**	**1**	**1**
11	**0**	**1**	**0**	**1**
10	**0**	**1**	**1**	**0**

$$\Rightarrow P_1 = \overline{a}_1.a_0.b_1 + a_1.\overline{a}_0.b_0 + a_0.b_1.\overline{b}_0 + a_1.\overline{b}_1.b_0$$
$$= a_0.b_1.\overline{\overline{(\overline{a}_1 + \overline{b}_O)}} + a_1.b_0.\overline{\overline{(\overline{a}_0 + \overline{b}_1)}}$$
$$= a_0.b_1.\overline{(a_1.b_0)} + a_1.b_0.\overline{(a_0.b_1)}$$
$$a_0.b_1 \oplus a_1.b_0$$

∗) **P_2**

a_1a_0 / b_1b_0	00	01	11	10
00	**0**	**0**	**0**	**0**
01	**0**	**0**	**0**	**0**
11	**0**	**0**	**0**	**1**
10	**0**	**0**	**1**	**1**

$$\Rightarrow P_2 = a_1.b_1.\overline{b}_0 + a_1.\overline{a}_0.b_1 = a_1.b_1.(\overline{a}_0 + \overline{b}_0)$$

∗) **P_3**

a_1a_0 / b_1b_0	00	01	11	10
00	**0**	**0**	**0**	**0**
01	**0**	**0**	**0**	**0**
11	**0**	**0**	**1**	**0**
10	**0**	**0**	**0**	**0**

$$\Rightarrow P_3 = a_1.a_0.b_1.b_0$$

∗) <u>Logigramme:</u>

<u>2ème méthode : Réalisation à l'aide des produits partiels</u>

			a_1		a_0
		*	b_1		b_0
			r_0		
	r_1		a_1b_0		a_0b_0
AC		**AC**		**AC**	
r_2					
	a_1b_1		a_0b_1		
P_3	P_2		P_1		P_0

∗) $P_0 = a_0.b_0$ et $r_0 = 0$

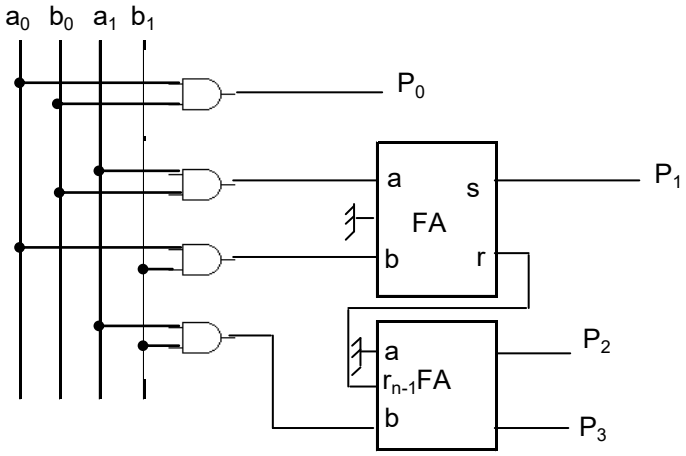
∗) $P_1 = a_0.b_1 \oplus a_1.b_0$ avec $r_1 = a_1.b_1.a_0.b_0$

∗) $P_2 = a_1.b_1 \oplus r_1 = a_1.b_1.\overline{a_1.b_1.a_0.b_0} + \overline{a_1.b_1}.a_1.b_1.a_0.b_0 = a_1.b_1.\overline{(a_0.b_0)}$

∗) $P_3 = a_1.b_1.r_1 = a_1.b_1.a_1.b_1.a_0.b_0 = a_1.b_1.a_0.b_0 = r_2$

Les produits partiels sont des produits arithmétiques qui sont identiques à des produits logiques, se réalisant donc à l'aide des portes AND.

- **)** Réaliser un multiplieur à l'aide des FA :



D. Comparateur :

Un comparateur est un circuit dont la fonction est d'indiquer, si un nombre binaire est inférieur, égal ou supérieur à un autre nombre binaire.

1/ Comparateur de deux nombres binaires à 1 bit :

Soient deux nombres binaires à 1 bit a et b, le comparateur permet d'affirmer que a est supérieur (S), inférieur (I) ou égal (E) à b.
Le tableau ci-dessous donne les différentes combinaisons possibles de la comparaison des deux nombres binaires a et b.

a	b	**E**	**S**	**I**
0	0	**1**	**0**	**0**
0	1	**0**	**0**	**1**
1	0	**0**	**1**	**0**
1	1	**1**	**0**	**0**

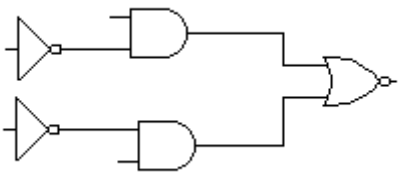
On a donc :

$$E = \overline{a}.\overline{b} + a.b = \overline{a \oplus b} = \overline{S + I}$$

$$S = a.\overline{b}$$

$$I = \overline{a}.b$$

Le circuit logique est donné par la figure ci-dessous :



2/ Comparateur de deux nombres binaires à 2 bits :

Pour comparer deux nombres binaires de 2 bits A et B, il faut effectuer une comparaison bit par bit, en commençant par les bits de poids plus fort. S'ils sont égaux, on passe aux bits de poids inférieur. On peut généraliser ceci pour un comparateur à N bits.

Application :

Réaliser un comparateur à 2 bits à l'aide d'un comparateur à 1 bit.